PRODUCT CONCEPT & IMPLEMENTATION BLUEPRINT

# Fault-Tolerant Multimodal RAG

A practical plan for turning resilient document search into a useful product for everyday people

**Core idea**
**A document assistant that can understand text, tables, charts, screenshots, and scanned pages—and recover gracefully when retrieval from ChromaDB fails or returns poor evidence.**

## What this document includes

- Product ideas and potential customer groups
- A short product description and project abstract
- System architecture and technology choices
- Fault-tolerant ChromaDB retrieval design
- A staged implementation plan, from text-only RAG to multimodal RAG
- Testing, evaluation, privacy, deployment, and product validation

Prepared as an implementation guide. Product concepts are hypotheses to validate with real users; reliability claims should be supported by testing.

# 1. Executive summary

The proposed product is a fault-tolerant multimodal Retrieval-Augmented Generation (RAG) assistant. A user uploads documents and asks questions in ordinary language. The system retrieves supporting evidence from text and visual content, then generates an answer with source references. If vector search fails, returns weak evidence, or cannot find a relevant image, the system attempts a controlled recovery path instead of silently producing an unsupported answer.

The product should not begin as a general-purpose assistant for everyone. Start with a focused use case where people already struggle to search complex documents—for example, students navigating course materials, households managing appliance manuals and bills, or freelancers searching client files. Validate one audience before broadening.

### Positioning

**“Ask your documents anything—and know when the system could not find a reliable answer.”**

The differentiator is not simply that it accepts images. It is that retrieval failures are detected, recovery is attempted, and uncertainty is communicated transparently.

# 2. Product ideas for everyday users

These are candidate products built on the same technical foundation. Choose one for the first release; the others can become later editions or separate market segments.

| Product angle | Who it helps / user problem | Example use | Why it could be compelling |
|---|---|---|---|
| **StudyMate: course-pack assistant** | Students who have lecture PDFs, screenshots, lab manuals, and diagrams spread across folders. | “Explain this circuit diagram and show which slide supports the answer.” | Clear recurring need; good fit for text, diagrams, tables, and citations. |
| **HomeDocs: household paperwork assistant** | People managing appliance manuals, warranty cards, bills, insurance documents, and receipts. | “Does this warranty cover this type of damage? Which page says so?” | Practical everyday use; source citations can reduce searching time. Avoid making legal or insurance decisions for users. |
| **FixIt Lens: manual and troubleshooting helper** | People trying to understand appliance manuals, labeled parts, warning icons, and troubleshooting charts. | “What does this warning light mean in the manual I uploaded?” | Naturally multimodal: text, diagrams, photos, and tables. Safety-critical repair advice needs safeguards. |
| **FormBuddy: forms and document explainer** | People who find application forms, instructions, and scanned documents confusing. | “What documents does this form ask me to attach?” | Useful for extraction and page citations; must protect personal information and avoid guaranteeing official outcomes. |
| **CreatorVault: searchable media library** | Creators, clubs, and small teams with screenshots, posters, briefs, PDFs, and visual references. | “Find the poster where we used the blue logo and the event date.” | Image and text retrieval together; team features could support paid plans. |
| **Small-business knowledge helper** | Local shops and solo operators with product catalogues, invoices, policies, and setup guides. | “Find the product spec and the page that lists its dimensions.” | Potential business subscription value; requires access control, tenant isolation, and dependable citations. |

**Best starting hypothesis: StudyMate or FixIt Lens. Both have easy-to-understand demos and naturally benefit from retrieving text plus diagrams. Interview target users before committing.**

# 3. Short product description

Fault-Tolerant Multimodal RAG is a document question-answering assistant that searches text, tables, charts, scanned pages, and images from a user's files. It uses ChromaDB for vector retrieval and includes recovery strategies for database errors, embedding failures, irrelevant search results, missing visual evidence, and timeouts. The assistant returns answers grounded in retrieved sources, or clearly states when the available evidence is insufficient.

# 4. Project abstract

People increasingly rely on complex digital documents containing a mixture of paragraphs, tables, charts, diagrams, screenshots, and scanned pages. Conventional keyword search and basic Retrieval-Augmented Generation systems may fail when query wording differs from the source, the vector database is unavailable, embeddings cannot be generated, or relevant visual evidence is not retrieved. This project proposes a fault-tolerant multimodal RAG system that accepts text-rich and image-rich documents, extracts and indexes their content, retrieves evidence through ChromaDB, and uses a vision-capable language model to generate source-grounded responses. A retrieval resilience layer detects transient failures, retries bounded operations, checks result relevance, and invokes configured fallback strategies such as keyword search, caption/OCR search, or a backup index. The system also validates that evidence is sufficient before answering and provides a controlled abstention when recovery fails. Reliability will be evaluated using retrieval quality, recovery success rate, answer faithfulness, latency, and correct abstention under simulated faults. The goal is to make document question answering more usable and resilient, while measuring rather than assuming improvements in reliability.

# 5. Product promise and boundaries

- **Promise:** Search mixed-format documents, recover from common retrieval failures, and show where answers came from.
- **Not a promise:** It will never hallucinate, always recover, or understand every image perfectly.
- **Trust feature:** Show source file, page number, and—where available—relevant text or image reference.
- **Safety boundary:** For medical, legal, financial, or safety-critical documents, present extracted evidence and limitations; do not claim to replace a qualified professional.

# 6. System architecture

## Recommended first version

Frontend: Streamlit. Backend orchestration: Python, with optional LangGraph after the initial pipeline is stable. Document processing: PyMuPDF for PDF text and page rendering, plus OCR for scanned pages. Embeddings: a chosen sentence-transformers model for text. Vector storage: ChromaDB. Keyword fallback: BM25 (for example, rank-bm25). Answering: Ollama with a vision-capable model that supports image input. Keep interfaces between components explicit so a model or retriever can be replaced.

| Layer | Responsibilities | Failure handling |
|---|---|---|
| Upload / validation | File type, size, page count, permissions, checksum | Reject unsupported files safely; keep per-file status |
| Ingestion | Text extraction, OCR, image/table extraction, chunking, metadata | Per-page recovery; retries; resumable jobs |
| Indexing | Text embeddings, image captions/OCR, Chroma collection, BM25 index | Model/version manifest; idempotent upserts; rebuild path |
| Retrieval | Query embedding, vector search, keyword search, candidate merge, rerank | Timeouts, bounded retries, fallback retriever, relevance gate |
| Answering | Vision LLM receives retrieved text and selected images | LLM retry/fallback; evidence validation; abstain if needed |
| Observability | Logs, metrics, fault traces, user-visible status | Record each failure and recovery result without logging sensitive content unnecessarily |

## Text-and-image retrieval design

- Store each text chunk with source file, page number, chunk ID, and document version.
- For every extracted image, store an image asset reference plus page/source metadata. Create a caption and OCR text when useful; index those text descriptions for initial image-aware retrieval.
- At query time, retrieve text chunks and image-related records. Deduplicate by source and page, then pass only relevant evidence to the vision model.
- Treat image captions as hints, not a substitute for the original image when the question depends on visual details.
- Later, add a dedicated multimodal embedding model and image-vector retrieval if the caption-based approach is insufficient.

## Important implementation note

**ChromaDB does not itself fix failed retrieval. The application must detect exceptions and poor results, choose fallbacks, and record what happened. A second collection in the same unavailable ChromaDB instance is not an independent fallback. For database-outage recovery, keep a separate BM25 index or a tested backup/replica.**

# 7. Retrieval fault-tolerance plan

Implement these mechanisms incrementally. Use bounded retries with exponential backoff and jitter for transient failures; do not retry configuration errors or invalid input indefinitely.

| Failure | Detection | Recovery | Important caveat |
|---|---|---|---|
| Embedding model unavailable | Timeout/exception, health check failure | Retry; then keyword retrieval if query text is available; optionally use a compatible, separately indexed backup | Different embedding spaces are not interchangeable; re-index if needed. |
| ChromaDB unavailable | Connection/query exception or timeout | Reconnect and retry; then search a separate BM25 index or verified replica | A fallback stored only in the same failed service is not independent. |
| No relevant results | Empty results or all relevance scores below calibrated threshold | Expand top-k; try keyword/hybrid search; then abstain if still unsupported | Similarity scores and thresholds need calibration. |
| Irrelevant results | Reranker/relevance gate says weak match; test-set signals | Hybrid search, query rewrite, reranking, metadata filters check | Do not claim retrieval is correct just because results exist. |
| Embedding/index mismatch | Manifest mismatch, dimension error, collection config mismatch | Stop incompatible search; select matching index or rebuild from source documents | Version model, dimensions, preprocessing, and collection. |
| Corrupted/stale index | Integrity checks, source/index counts, failed reads, document version mismatch | Re-index affected files idempotently; rebuild from source-of-truth documents | Maintain source files and a recovery manifest. |
| Image retrieval misses | No image candidates or weak caption/OCR match | Search OCR/captions/page text; retrieve surrounding page; optionally use image embeddings | Keep original image path and page mapping. |
| Slow/timed-out search | Deadline exceeded, latency above SLO | Timeout; one bounded retry if safe; keyword fallback; return partial status | Set an overall request deadline to prevent retry storms. |

## Suggested retrieval controller

1. Validate query and collection/index manifest. 2. Try vector retrieval with a strict timeout. 3. If the vector path fails technically, use the independent keyword index. 4. If it returns weak results, merge vector and keyword candidates, then rerank. 5. For visual questions, add caption/OCR/page-level candidates. 6. Check evidence sufficiency. 7. Return evidence or abstain. Log the failure category, recovery action, latency, and outcome.

# 8. Full implementation roadmap

Build a working, testable text-only baseline before introducing images. This isolates retrieval faults from multimodal processing faults and makes debugging easier.

## Phase 0 — Scope and baseline

- Choose one target audience and a narrow first use case.
- Collect a small, permission-cleared set of sample PDFs and write 30–50 questions with expected source pages.
- Record baseline Recall@5, MRR, answer faithfulness, latency, and correct abstention.

## Phase 1 — Text-only RAG

- Create a Python project and virtual environment; pin dependencies.
- Load PDFs with PyMuPDF/PyPDF; split into chunks with overlap.
- Generate text embeddings; store chunks and metadata in ChromaDB.
- Implement top-k similarity search and an LLM answer prompt that includes retrieved evidence.
- Return source file and page references; abstain when evidence is insufficient.

## Phase 2 — Retrieval fault tolerance

- Wrap embedding and Chroma operations with typed error handling, timeouts, bounded retries, and exponential backoff.
- Build a separate BM25 index from the same chunks and metadata.
- Implement fallback routing for embedding failure and ChromaDB outage.
- Add a relevance gate; calibrate thresholds using labeled questions rather than arbitrary values.
- Add hybrid candidate merging and a reranker; test index/model manifests and rebuild logic.
- Log faults and recovery results; create a fault-injection test suite.

## Phase 3 — Reliable answer generation

- Use a supported local or hosted LLM; keep the model interface replaceable.
- Add answer timeout/retry, optional model fallback, output validation, citation checks, and safe abstention.
- Separate retrieval failures from generation failures in logs and metrics.
- Avoid claiming citations prove correctness; verify cited passages support the answer.

## Phase 4 — Multimodal ingestion

- Render PDF pages and extract embedded images with page/source mapping.
- Detect scanned pages and use OCR as a fallback when text extraction is empty or weak.
- Extract tables with a table-aware parser where needed; preserve row/column structure when possible.
- Generate concise image captions with a vision model; retain original images for later reasoning.
- Store image metadata, captions, OCR text, file hashes, and processing status.
- Make ingestion resumable and idempotent so a failed page can be retried without duplicating the whole index.

## Phase 5 — Multimodal retrieval and answering

- Index captions/OCR alongside text chunks for a first image-aware retrieval approach.
- Retrieve both text and image references; deduplicate and rerank candidates.
- For questions that depend on visual details, send the original retrieved image plus relevant text to a vision-capable LLM.
- If image retrieval fails, try caption/OCR and surrounding-page retrieval; if visual evidence remains unavailable, say so.

- Later, evaluate a dedicated multimodal embedding model and image-vector retrieval.

## Phase 6 — UI, deployment, and security

- Build a Streamlit interface for uploads, questions, source citations, and visible recovery messages.
- Add file limits, type validation, malware scanning appropriate to the deployment, and safe temporary-file handling.
- Protect user files with authentication, access controls, per-user indexes, and deletion workflows.
- Containerize only after local tests pass; add health checks, structured logs, and configuration through environment variables.
- Do not expose API keys or log private document contents by default.

## Phase 7 — Evaluate and iterate

- Inject failures: stop ChromaDB, simulate embedding timeouts, corrupt a page, remove an image, and return empty/irrelevant candidates.
- Measure recovery success rate, retrieval quality, correct abstention, end-to-end success, latency, and cost.
- Compare baseline vs recovery-enabled system; report trade-offs and failure cases honestly.
- Pilot with 5–10 target users; observe whether citations and recovery messages are understandable.

# 9. Suggested project structure

```
multimodal-rag/ app.py # Streamlit UI config.py # Models, paths, timeouts, thresholds ingestion/ pdf_loader.py # Text extraction and page mapping ocr.py # OCR fallback image_extractor.py # Extract images and render pages chunking.py # Text/table chunking indexing/ embeddings.py # Embedding model wrapper + manifest chroma_store.py # Chroma operations bm25_store.py # Independent keyword fallback index index_manifest.json # Model/version/source metadata retrieval/ controller.py # Recovery routing and deadlines hybrid.py # Candidate merge reranker.py # Optional reranking relevance.py # Evidence sufficiency gate answering/ llm_client.py # Vision/text model wrapper validation.py # Citation and output checks evaluation/ golden_questions.json # Questions and expected sources fault_injection.py # Simulated failures metrics.py # Recall, MRR, latency, recovery rate data/ uploads/ # User source files (not committed) indexes/ # Local indexes (not committed) tests/ test_ingestion.py test_retrieval.py test_fallbacks.py test_multimodal.py
```

# 10. Evaluation plan

| Metric | What it measures | How to test |
|---|---|---|
| Recall@5 | Whether a relevant chunk appears among the top five | Compare retrieved chunk IDs against labeled relevant chunks |
| MRR | How highly the first relevant result ranks | Use questions with known relevant source chunks |
| Recovery success rate | How often the system completes retrieval after an injected failure | Repeat tests with each failure class; count successful evidence retrieval |
| Correct abstention rate | Whether it refuses when evidence is insufficient | Ask questions whose answers are absent from the corpus |
| Answer faithfulness | Whether claims are supported by retrieved evidence | Human review or a validated evaluation protocol |
| Latency | Response time and recovery overhead | Measure p50/p95 under normal and failure conditions |

## Initial acceptance criteria (targets to revise after baseline)

- At least 90% of defined transient-failure test cases recover or fail gracefully without crashing.
- No answer is returned as grounded when the evidence check fails; correct abstention is measured separately.
- Every returned answer has source references where evidence was retrieved and citations can be verified.
- Recovery overhead and p95 latency are measured and documented.

These are proposed engineering targets, not measured results. Tune them based on the corpus, hardware, and user expectations.

# 11. How to sell it to everyday users

People usually buy an outcome, not the term “multimodal RAG.” Lead with the problem solved and show a short demo.

| Audience | Plain-language pitch | Possible pricing model to test |
|---|---|---|
| Students | “Ask your PDFs, notes, diagrams, and lab manuals. Get an answer with page references.” | Free tier; student premium for larger libraries or study features. |
| Households | “Find answers in your manuals, bills, warranties, and receipts without digging through folders.” | Low-cost personal plan or local-only desktop version. |
| DIY / appliance owners | “Ask a manual what a warning icon means and see the page it came from.” | One-time purchase, subscription, or manufacturer partnership. |
| Creators / clubs | “Search posters, screenshots, briefs, and event documents by what they contain.” | Per-seat or shared-library subscription. |
| Small businesses | “Find the right product spec, policy, or instruction from your business files—with sources.” | Monthly team plan based on users and indexed storage. |

## Go-to-market experiments

- Build a 60-second demo showing one text question, one chart/image question, and one deliberately simulated retrieval failure that recovers.
- Interview 5–10 people in one audience. Ask what documents they search, how often, and what a wrong answer would cost them.
- Test a free local prototype before building payments, team sharing, or a cloud service.
- Use trust as the differentiator: show sources, explain failed retrieval, and do not invent an answer when evidence is missing.
- Avoid broad “works for every document” claims until you have evaluated diverse PDFs and images.

# 12. Risks and mitigations

| Risk | Mitigation |
|---|---|
| Vision model misreads a chart | Show source image/page; test chart-specific questions; ask for clarification when labels are unclear. |
| Fallback returns stale or weak evidence | Version indexes; validate source freshness; apply the same relevance checks to every fallback. |
| Retries increase cost and latency | Bound retries, enforce deadlines, use backoff, and expose recovery status. |
| Sensitive documents are exposed | Prefer local processing when appropriate; implement access control, deletion, encryption, and careful logging. |
| Users over-trust the answer | Provide citations and uncertainty messages; state limitations; evaluate faithfulness. |

# 13. Recommended MVP

**For the first demo, build a local StudyMate-style assistant that accepts PDFs with text and images. Extract text and page images, index text and image captions/OCR in ChromaDB, maintain a separate BM25 fallback, retrieve and rerank candidates, and pass selected text plus original images to a vision-capable Ollama model. Include source citations, bounded retries, a retrieval relevance gate, and an explicit “could not find reliable evidence” response. Evaluate normal cases and injected retrieval failures before adding audio/video or multi-user cloud features.**

# 14. Final implementation checklist

- ■ Select one audience and create a labeled question set.
- ■ Build text-only RAG with page-level citations.
- ■ Add ChromaDB error handling and independent BM25 fallback.
- ■ Add relevance checks, hybrid retrieval, and optional reranking.
- ■ Track embedding model and index versions; test rebuild procedures.
- ■ Add answer retries, validation, and safe abstention.
- ■ Add OCR, page rendering, image extraction, captions, and image metadata.
- ■ Retrieve image references and pass selected original images to a vision model.
- ■ Inject failures and record recovery rate, faithfulness, latency, and correct abstention.
- ■ Pilot with target users and revise the product based on observed needs.

# 15. Technical references

Use official documentation to confirm APIs and model support before implementation; versions change.

- Chroma documentation: https://docs.trychroma.com/
- LangGraph documentation: https://docs.langchain.com/oss/python/langgraph/overview
- Ollama vision documentation: https://ollama.com/blog/vision-models
- PyMuPDF documentation: https://pymupdf.readthedocs.io/
- Sentence Transformers documentation: https://www.sbert.net/
- rank-bm25 project: https://github.com/dorianbrown/rank_bm25

This blueprint is a proposed design. It does not claim that the system has been implemented, benchmarked, or validated. Verify model licenses, data privacy obligations, and deployment requirements for your intended audience.